connected, and 'false' otherwise

- *is_planar(<g>)* - returns 'true' if graph <g> can be placed on a plane without its edges crossing each other, and 'false' otherwise
- *is_tree(<g>)* - returns 'true' if graph <g> has no simple cycles, and 'false' otherwise
- *is_biconnected(<g>)* - returns 'true' if graph <g> will remain connected even after removal of any one of its vertices and the edges incident on that vertex, and 'false' otherwise
- *is_bipartite(<g>)* - returns 'true' if graph <g> is bipartite, i.e., two-colourable, and false otherwise
- *is_isomorphic(<g1>, <g2>)* - returns 'true' if graphs <g1> and <g2> have the same number of vertices and are connected in the same way, and 'false' otherwise. And, isomorphism (<g1>, <g2>) returns an isomorphism (that is a one-to-one onto mapping) between the graphs <g1> and <g2>, if it exists.
- *is_edge_in_graph(<edge>, <g>)* - returns 'true' if <edge> is in graph <g>, and 'false' otherwise
- *is_vertex_in_graph(<vertex>, <g>)* - returns 'true' if <vertex> is in graph <g>, and 'false' otherwise

The following example specifically demonstrates the isomorphism property, from the list above:

```
$ maxima -q
(%i1) load(graphs)$
...
0 errors, 0 warnings
(%i2) g1: create_graph(3, [[0, 1], [0, 2]]);
(%o2)                    GRAPH(3 vertices, 2 edges)
(%i3) g2: create_graph(3, [[1, 2], [0, 2]]);
(%o3)                    GRAPH(3 vertices, 2 edges)
(%i4) is_isomorphic(g1, g2);
(%o4)                             true
(%i5) isomorphism(g1, g2);
(%o5)                      [2 -> 0, 1 -> 1, 0 -> 2]
(%i6) quit();
```

Graph neighbourhoods

A lot of the properties of graphs are linked to vertex and edge neighbourhoods, also referred to as adjacencies.

For example, a graph itself could be represented by an adjacency list or matrix, which specifies the vertices adjacent to the various vertices in the graph. *adjacency_matrix(<g>)* returns the adjacency matrix of the graph <g>.

The number of edges incident on a vertex is called the valency or degree of the vertex, and could be obtained using *vertex_degree(<v>, <g>)*. *degree_sequence(<g>)* returns the list of degrees of all the vertices of the graph <g>. In case of a directed graph, the degrees could be segregated as in-degree and out-degree, as per the edges incident into and out of the vertex, respectively. *vertex_in_degree(<v>, <dg>)* and *vertex_out_degree(<v>, <dg>)*, respectively, return the in-degree and out-degree for the vertex <v> of the directed graph <dg>.

neighbors(<v>, <g>), *in_neighbors(<v>, <dg>)* and *out_neighbors(<v>, <dg>)* return the list of adjacent vertices, adjacent in-vertices and the adjacent out-vertices, respectively, of the vertex <v> in the corresponding graphs.

average_degree(<g>) computes the average of the degrees of all the vertices of the graph <g>. *max_degree(<g>)* finds the maximal degree of vertices of the graph <g>, and returns one such vertex alongwith. *min_degree(<g>)* finds the minimal degree of vertices of the graph <g>, and returns one such vertex alongwith.

Here follows a neighbourhood related demonstration:

```
$ maxima -q
(%i1) load(graphs)$
...
0 errors, 0 warnings
(%i2) g: create_graph(4, [[0, 1], [0, 2], [0, 3], [1, 2]]);
(%o2)                    GRAPH(4 vertices, 4 edges)
(%i3) string(adjacency_matrix(g)); /* string for output in single
line */
(%o3)          matrix([0,0,0,1],[0,0,1,1],[0,1,0,1],[1,1,1,0])
(%i4) degree_sequence(g);
(%o4)                          [1, 2, 2, 3]
(%i5) average_degree(g);
(%o5)                                2
(%i6) neighbors(0, g);
(%o6)                          [3, 2, 1]
(%i7) quit();
```

Graph connectivity

A graph is ultimately about connections, and hence lots of graph properties are centred around connectivity.

vertex_connectivity(<g>) returns the minimum number of vertices that need to be removed from the graph <g> to make the graph <g> disconnected. Similarly, *edge_connectivity(<g>)* returns the minimum number of edges that need to be removed from the graph <g> to make the graph <g> disconnected.

vertex_distance(<u>, <v>, <g>) returns the length of the shortest path between the vertices <u> and <v> in the graph <g>. The actual path could be obtained using *shortest_path(<u>, <v>, <g>)*.

girth(<g>) returns the length of the shortest cycle in graph <g>.

vertex_eccentricity(<v>, <g>) returns the maximum of the vertex distances of vertex <v> with any other vertex in the connected graph <g>.

diameter(<g>) returns the maximum of the vertex eccentricities of all the vertices in the connected graph <g>.

radius(<g>) returns the minimum of the vertex eccentricities of all the vertices in the connected graph <g>.

graph_center(<g>) returns the list of vertices that have eccentricities equal to the radius of the connected graph <g>.

graph_periphery(<g>) is the list of vertices that have eccentricities equal to the diameter of the connected graph.

A minimal connectivity related demonstration for the graph shown in Figure 1 follows:

```
$ maxima -q
(%i1) load(graphs)$
```